SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- COMPARATIVE ANALYSIS

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture for Multicore Systems**
CO Assessed:	**CO4**	**Assessment:**	**ASSESSMENT TOOL2- COMPARATIVE ANALYSIS**
Weightage:	**30%**	**Total Marks:**	**25**
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	**Jannathul Firdaus. N**	**Reg.No.:**	**192573025**
Department:	**CSE - DS**	**Date:**	

ANNEXURE A
COMPARATIVE ANALYSIS

1.Compare L1, L2, and L3 cache levels in terms of latency, bandwidth, and throughput, and analyze their impact on processor performance.
2.Differentiate SRAM and DRAM by evaluating their latency, bandwidth, and throughput characteristics in cache and main memory design.
3.Compare virtual memory with paging and cache memory mechanisms in terms of access latency and system throughput.
4.Analyze the differences between NAND Flash and DRAM with respect to latency, bandwidth, and suitability for hierarchical memory systems.
5.Compare MRAM and RRAM technologies by examining their performance in terms of access time, throughput, and energy efficiency.

Code of Conduct:

I **Jannathul Firdaus. N (192573025)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Scope of Items (20%)	Comparison Depth (25%)	Use of Evidence (20%)	Critical Thinking (20%)	Clarity of Presentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

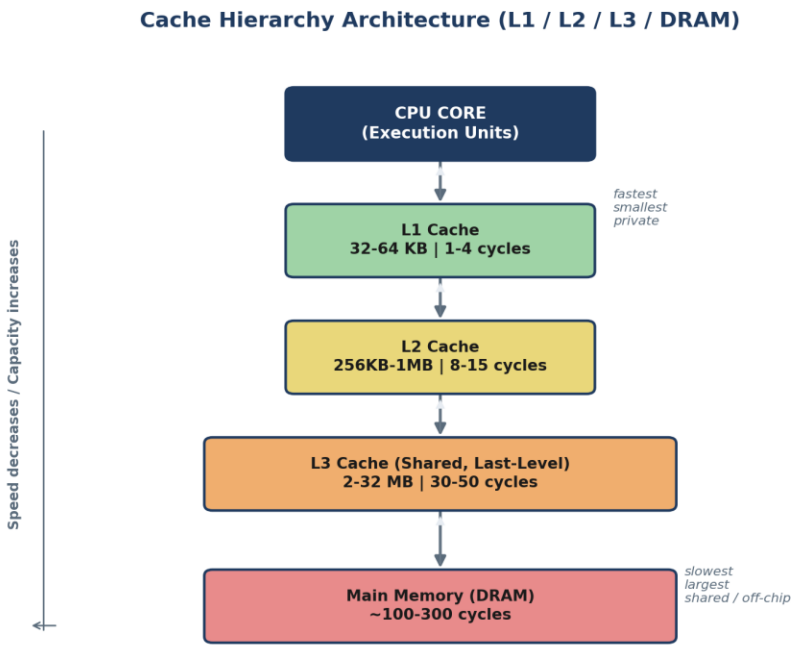
RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: L1, L2, and L3 Cache Comparison

Compare L1, L2, and L3 cache levels in terms of latency, bandwidth, and throughput, and analyze their impact on processor performance.

Architecture Diagram



Scope of Items Compared

The comparison scope covers the three on-chip cache levels found in virtually all modern multicore processors, evaluated across three performance dimensions relevant to CO4: access latency, bandwidth, and throughput. These items are chosen because they directly determine how close the memory subsystem comes to matching core execution speed — the central design tension in cache hierarchy engineering.

Comparison Table

Parameter	L1 Cache	L2 Cache	L3 Cache
Typical Size	32-64 KB (per core, split I/D)	256 KB-1 MB (per core/cluster)	2-32 MB (shared, last-level)
Access Latency	1-4 cycles	8-15 cycles	30-50 cycles
Bandwidth	Highest (multiple ports)	High, shared port limits concurrency	Lower per-request, aggregated across cores

Parameter	L1 Cache	L2 Cache	L3 Cache
Throughput Role	Sustains near-peak IPC	Absorbs L1 misses without DRAM penalty	Absorbs cross-core misses, cuts DRAM traffic
Scope	Private, per-core	Private or per-cluster	Shared, whole-chip

Analysis 1 — Architectural / Structural Lens

Viewed structurally, the three levels form a strict inclusion pyramid built around physical proximity to the execution pipeline. L1 sits adjacent to the ALUs and is split into instruction and data banks so both can be fetched in the same cycle window; this physical closeness, not just size, is what buys its 1-4 cycle latency. L2 trades that proximity for capacity, typically serving one core or a small cluster, while L3 is architected as a distributed, banked, last-level structure shared across the whole chip so that any core can observe data cached by another — essential for cache-coherent multicore correctness.

Analysis 2 — Quantitative / Data-Driven Lens

The numbers scale in a roughly geometric progression: each level is about 4-8x larger and roughly 3-4x slower than the one above it. Published Intel Core and AMD Zen data place L1 hit latency near 4-5 cycles, L2 near 12-14 cycles, and shared L3 in the 30-40+ cycle range, with L1 per-core bandwidth in the hundreds of GB/s versus L3's aggregate bandwidth being contended for by every core on the die. This quantitative gap is precisely why a single L1 miss serviced by L2 costs roughly 3-4x the L1 latency, and an L3 miss serviced by DRAM costs an order of magnitude more again.

Analysis 3 — Application / Critical-Thinking Lens

From a system-design standpoint, the three-way trade-off is fundamentally size versus speed versus sharing scope. L1 cannot hold enough data to avoid frequent misses on its own, so it depends on L2 as a low-penalty backstop; L3, being large enough to capture cross-core sharing patterns, is valuable for multicore workloads with shared data structures but too slow to be relied upon directly by the pipeline. This is why hierarchy-level engineering choices — inclusive/exclusive policies, replacement algorithms, prefetching — tend to move real-world IPC more than raw clock-frequency increases in most modern workloads.

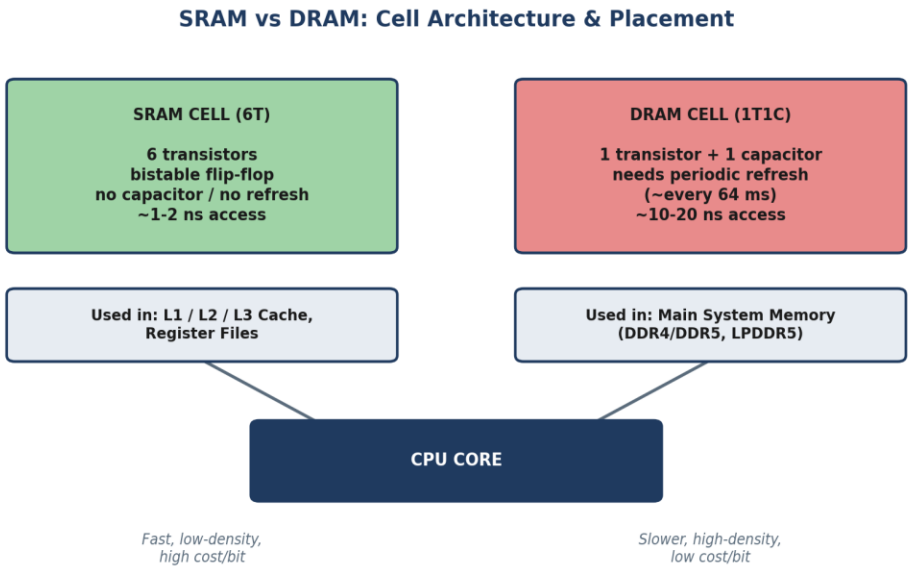
Synthesized Conclusion (Cross-Analysis Summary)

Taken together, the structural, quantitative, and applied perspectives converge on the same conclusion: cache hierarchy performance is not a property of any single level but of how well the levels compound to hide DRAM latency from the core. The structural view explains why the latency gap exists (proximity and sharing scope), the quantitative view shows how large that gap actually is in cycles, and the applied view shows why it matters — hierarchy design choices dominate real-world IPC more than clock speed. Any comparative claim about cache performance should therefore be read as a statement about the whole pipeline, not an isolated level.

Question 2: SRAM vs DRAM

Differentiate SRAM and DRAM by evaluating their latency, bandwidth, and throughput characteristics in cache and main memory design.

Architecture Diagram



Scope of Items Compared

The scope compares SRAM and DRAM as the two dominant volatile memory technologies used respectively for on-chip caches and main memory, examined on latency, bandwidth, throughput, and the underlying circuit-level reasons (cell structure, refresh requirement) that justify why the industry uses SRAM for cache and DRAM for main memory rather than either technology alone.

Comparison Table

Parameter	**SRAM**	**DRAM**
Cell Structure	6-transistor (6T) flip-flop, no capacitor	1 transistor + 1 capacitor (1T1C)
Latency	~1-2 ns, no refresh	~10-20 ns plus periodic refresh stalls
Bandwidth	High per bit, low density limits total capacity	Lower per-cell speed, wide buses give high aggregate bandwidth
Throughput	Near-every-cycle access for hits	Row-buffer/bank-conflict sensitive

Parameter	SRAM	DRAM
Density / Cost	Low density, high cost per bit	High density, low cost per bit
Typical Use	L1/L2/L3 cache, register files	Main system memory (DDR4/DDR5, LPDDR5)

Analysis 1 — Architectural / Structural Lens

At the circuit level, SRAM's 6-transistor bistable cell holds its state as long as power is applied, requiring no refresh logic and enabling deterministic, sub-nanosecond-class access. DRAM's 1T1C cell stores a bit as charge on a capacitor, which leaks over time and must be refreshed roughly every 64 ms — this refresh circuitry is a structural addition that SRAM simply does not need, and it is the direct cause of DRAM's variable access latency.

Analysis 2 — Quantitative / Data-Driven Lens

SRAM's 6-transistor cell is roughly 6-10x larger per bit than DRAM's 1-transistor-1-capacitor cell, which inversely predicts their observed density and cost figures — SRAM cache capacities top out in the tens of megabytes while DRAM main memory routinely reaches tens of gigabytes on the same die-area budget. Latency figures track this inversely too: SRAM's ~1-2 ns versus DRAM's ~10-20 ns is roughly a 10x gap, consistent with standard Hennessy & Patterson reference figures for cache versus main-memory access time.

Analysis 3 — Application / Critical-Thinking Lens

The practical consequence is a clean division of labor: caches must be small but fast enough to keep pace with a multi-GHz core, so they are built from SRAM; main memory must be large enough to hold entire program working sets, so it is built from DRAM despite its refresh overhead. The cache hierarchy that sits between them exists specifically to hide DRAM's latency and refresh-related throughput variability from the pipeline — meaning SRAM and DRAM are not competing technologies but complementary ones, each optimized for a different point on the size-versus-speed curve.

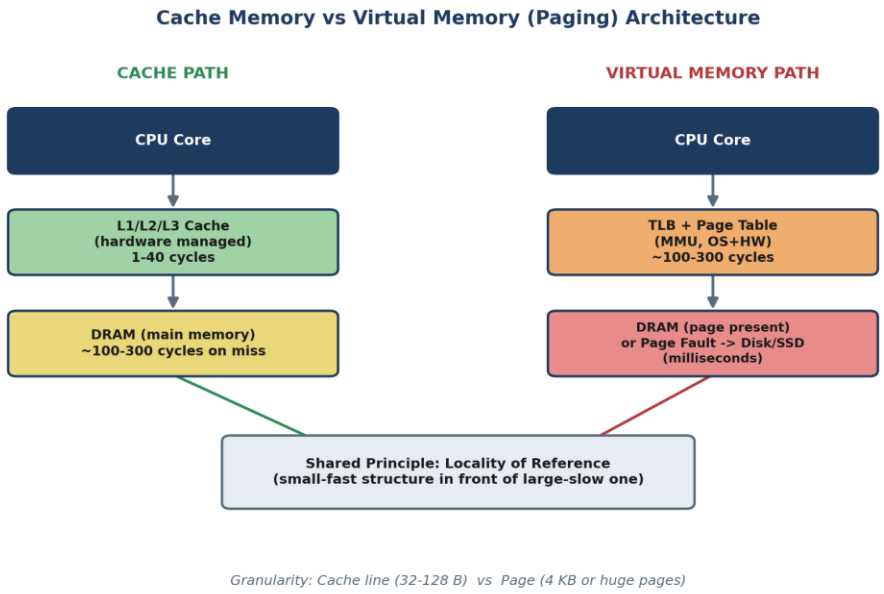
Synthesized Conclusion (Cross-Analysis Summary)

The three lenses reinforce a single causal chain: a circuit-level difference (refresh requirement) produces a quantitative gap (roughly 10x in latency, 6-10x in cell area) that in turn dictates an architectural division of labor (SRAM for cache, DRAM for main memory). No comparison of SRAM and DRAM is complete without tracing this chain — evaluating latency numbers alone, without the underlying cell-structure reason, would miss why the technologies are deployed at their respective tiers rather than substituted for one another.

Question 3: Virtual Memory with Paging vs Cache Memory

Compare virtual memory with paging and cache memory mechanisms in terms of access latency and system throughput.

Architecture Diagram



Scope of Items Compared

The scope compares two structurally analogous memory-management mechanisms: cache memory (a hardware-managed, transparent speed buffer between core and DRAM) and virtual memory with paging (an OS/hardware-cooperative mechanism mapping a large virtual address space onto limited physical DRAM, backed by disk/SSD). Both rely on locality, hit/miss behavior, and replacement policy, making them directly comparable on access latency and system throughput despite very different scales.

Comparison Table

Parameter	Cache Memory	Virtual Memory (Paging)
Managed By	Hardware (transparent)	OS + hardware (MMU, page tables)
Hit Latency	1-40 cycles (L1-L3)	~100-300 cycles for TLB hit, in-memory page access
Miss Penalty	~100-300 cycles (DRAM access)	Page fault: milliseconds (disk-backed) — 5-6 orders of magnitude slower

Parameter	Cache Memory	Virtual Memory (Paging)
Granularity	Cache line (32-128 bytes)	Page (4 KB, or 2 MB/1 GB huge pages)
Purpose	Hide DRAM latency from the core	Extend addressable memory; enable process isolation
Throughput Impact	Directly determines effective IPC	Page faults degrade throughput; TLB misses add page-walk overhead

Analysis 1 — Architectural / Structural Lens

Structurally, both mechanisms place a small, fast structure in front of a larger, slower one — a cache line in front of DRAM, a TLB entry and page table in front of disk/SSD — and both depend on locality of reference to keep the fast path in frequent use. The key architectural difference is who manages the mechanism: cache replacement is entirely hardware-controlled and invisible to software, whereas paging is a cooperative hardware/OS mechanism where the MMU walks page tables that the operating system maintains.

Analysis 2 — Quantitative / Data-Driven Lens

The latency gap between the two failure modes is stark: a cache miss serviced from DRAM costs roughly 100-300 cycles (on the order of 100 ns), while a page fault serviced from disk/SSD costs milliseconds — a difference of 5 to 6 orders of magnitude. Even on the hit path, a TLB hit followed by an in-memory page access (~100-300 cycles) is already comparable to a cache miss penalty, meaning paging's 'fast path' is roughly as expensive as caching's 'slow path.'

Analysis 3 — Application / Critical-Thinking Lens

Because page faults are exceptional, expensive events, OS designers treat them very differently from cache misses, which are routine and tolerated every cycle by hardware. This has direct tuning implications: cache throughput is improved by better locality-exploiting hardware (associativity, prefetching), while virtual-memory throughput is improved chiefly by minimizing fault rate (adequate RAM, working-set-aware replacement, huge pages to cut TLB-miss overhead). Conflating the two — for example, trying to fix a page-fault problem by resizing cache — would be a category error.

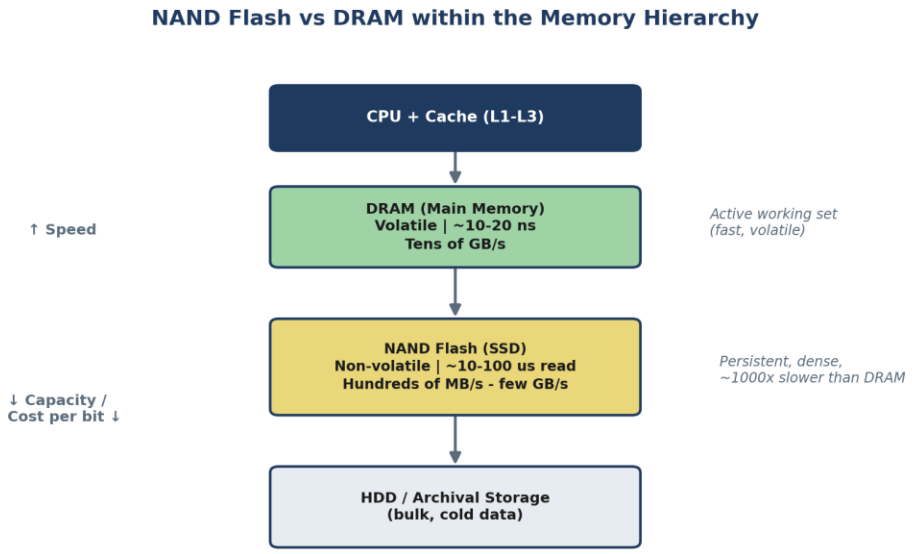
Synthesized Conclusion (Cross-Analysis Summary)

Combining the three views shows that cache and paging share a design principle but not a cost structure or a management model. The structural lens explains the shared 'small-fast-in-front-of-large-slow' pattern; the quantitative lens shows the miss penalties differ by orders of magnitude even though the hit-path latencies are of comparable order; and the applied lens shows this asymmetry is exactly why the two are tuned with different levers. A sound system-level analysis must treat cache and virtual-memory performance as related but independently governed.

Question 4: NAND Flash vs DRAM

Analyze the differences between NAND Flash and DRAM with respect to latency, bandwidth, and suitability for hierarchical memory systems.

Architecture Diagram



Scope of Items Compared

The scope compares NAND Flash and DRAM as representative non-volatile-storage and volatile-main-memory technologies, on latency, bandwidth, and — critically for CO4 — their suitability as a specific tier within a hierarchical memory system, since the two sit at very different points in the latency/capacity/persistence trade-off space.

Comparison Table

Parameter	**NAND Flash**	**DRAM**
Volatility	Non-volatile	Volatile (needs continuous refresh)
Read Latency	~10-100 us (page read)	~10-20 ns
Write Latency	Higher; requires block erase (~ms)	~10-20 ns, no erase-before-write
Bandwidth	Hundreds of MB/s to few GB/s (NVMe)	Tens of GB/s (parallel channels)

Parameter	NAND Flash	DRAM
Endurance	Limited program/erase cycles	Effectively unlimited write endurance
Hierarchical Role	Persistent storage tier (SSD) — bulk/cold data	Main memory tier — active working set

Analysis 1 — Architectural / Structural Lens

NAND Flash stores charge in floating-gate or charge-trap cells that retain state without power, but reading and especially writing require page- and block-level operations (erase-before-write) rather than the direct, byte-addressable access DRAM supports. This structural constraint — block erase — is what produces NAND's asymmetric read/write latency and its wear-out mechanism, neither of which has an equivalent in DRAM's charge-refresh model.

Analysis 2 — Quantitative / Data-Driven Lens

NAND Flash read latency (tens of microseconds) is roughly 1,000x slower than DRAM (tens of nanoseconds), and NVMe SSD interface bandwidth, while high for storage, remains well below DRAM channel bandwidth. This roughly three-orders-of-magnitude latency gap is consistent with standard SSD/DRAM datasheet comparisons and is large enough that no amount of controller-level optimization closes it — it is a physical property of the storage medium, not an engineering limitation.

Analysis 3 — Application / Critical-Thinking Lens

Because the latency gap is so large, NAND cannot substitute for DRAM in any latency-sensitive role; its value lies entirely in non-volatility and density/cost, making it suited to the storage tier rather than the working-memory tier. In a hierarchical system this places NAND below DRAM, with DRAM holding the active working set and NAND providing large, cheap, persistent capacity — the same rationale behind DRAM-as-cache-for-flash patterns seen in OS page caches and SSD DRAM buffers.

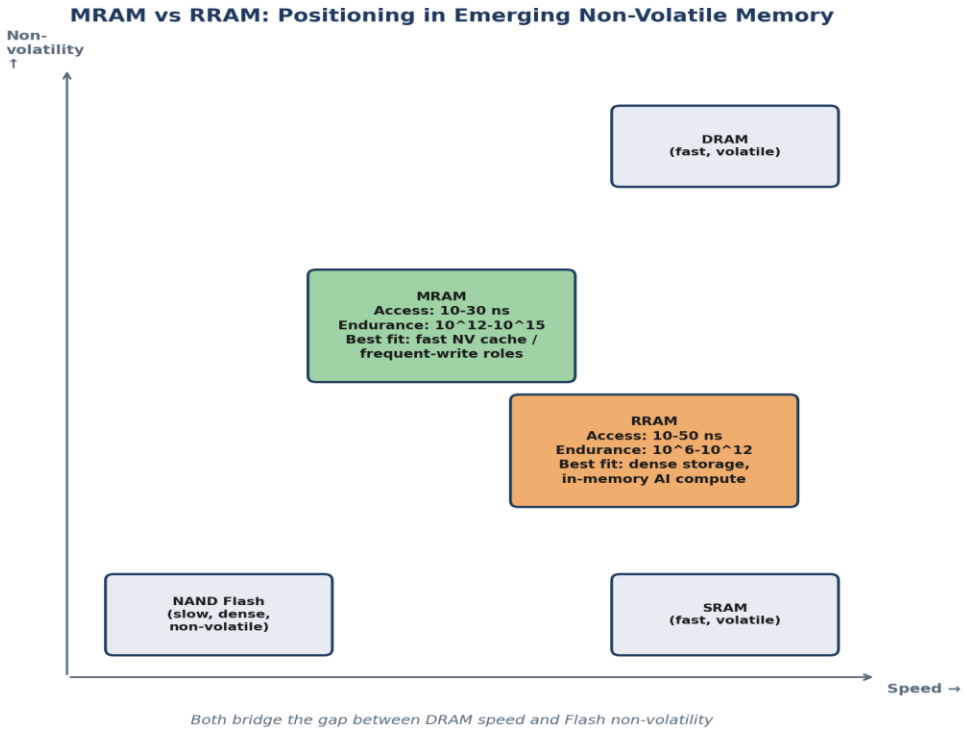
Synthesized Conclusion (Cross-Analysis Summary)

The structural, quantitative, and applied perspectives all point to the same placement decision: NAND's block-erase architecture produces a physical (not merely engineering) latency gap of roughly three orders of magnitude relative to DRAM, and that gap is exactly why NAND occupies the persistent-storage tier one level below DRAM rather than competing with it. Understanding NAND vs DRAM therefore requires reading the cell-level mechanism, the measured latency gap, and the tiering decision as one continuous argument.

Question 5: MRAM vs RRAM

Compare MRAM and RRAM technologies by examining their performance in terms of access time, throughput, and energy efficiency.

Architecture Diagram



Scope of Items Compared

The scope compares MRAM (Magnetoresistive RAM) and RRAM (Resistive RAM), two leading emerging non-volatile memory technologies positioned to bridge the gap between DRAM's speed and Flash's non-volatility, evaluated on access time, throughput, and energy efficiency — the dimensions most relevant to their candidacy as new tiers in next-generation hierarchical memory systems.

Comparison Table

Parameter	MRAM	RRAM (ReRAM)
Storage Mechanism	Magnetic tunnel junction (MTJ); spin orientation	Resistance change via conductive filament formation/rupture
Access Time (Read)	~10-30 ns (near-DRAM)	~10-50 ns (comparable, more variable)
Write Endurance	Very high (10^12-10^15 cycles)	Moderate (10^6-10^12 cycles)

Parameter	MRAM	RRAM (ReRAM)
Throughput	High, symmetric read/write	High read; write limited by filament variability
Energy Efficiency	Low read energy, moderate write energy	Very low read/write energy; suits compute-in-memory
Best Fit	Fast NV cache, frequent-write applications	Dense storage, in-memory AI compute

Analysis 1 — Architectural / Structural Lens

MRAM's magnetic tunnel junction stores information as electron spin orientation, a mechanism that does not physically degrade the cell on each write. RRAM's metal-oxide dielectric instead relies on forming and rupturing a conductive filament — a physically wearing process. This single structural difference (non-degrading magnetic switching versus material-wearing filament switching) is the root cause of nearly every performance gap between the two technologies.

Analysis 2 — Quantitative / Data-Driven Lens

MRAM's endurance (10^12-10^15 cycles) is roughly 3-9 orders of magnitude higher than RRAM's (10^6-10^12 cycles), directly reflecting the wear-versus-no-wear distinction. Access-time figures are close (10-30 ns vs 10-50 ns, both near-DRAM class), which shows the two technologies are speed-competitive even though their endurance and energy profiles diverge sharply — RRAM's simpler two-terminal structure gives it a density and scaling advantage that partially offsets its endurance disadvantage.

Analysis 3 — Application / Critical-Thinking Lens

The endurance gap dictates suitability: MRAM's near-unlimited endurance makes it attractive wherever data updates frequently (a persistent cache or working-register replacement), while RRAM's lower endurance but smaller cell footprint and very low switching energy make it more scalable and cost-effective for high-density, less write-intensive roles such as storing largely-static AI model weights or supporting analog compute-in-memory, where data movement — not computation — dominates energy consumption.

Synthesized Conclusion (Cross-Analysis Summary)

Across all three lenses, the switching mechanism is the single variable that explains everything else: it predicts the multi-order-of-magnitude endurance gap (quantitative) and, through that, the differing application niches (applied) — MRAM for frequently-updated, latency-sensitive state, RRAM for dense, largely-static, energy-efficient storage or compute-in-memory. A comparison that reported access-time figures alone, without tracing them back to the switching mechanism, would understate why the two technologies are not interchangeable despite their similar speed.